

WEEK 5-PAYMENT TRANSACTION MANAGEMENT SYSTEM

Movie Ticket Booking Management System

1. Scenario:

The Movie Ticket Booking Management System needs to maintain payment information for customer movie bookings. The system records payment details, tracks transaction status, manages successful and failed payments, and generates payment-related business reports. The Payment Transaction Module is connected with the existing Customer and Bookings modules. This helps the system maintain accurate payment records for movie ticket bookings.

2. Create Payment Table:

Payment Entity:

The PAYMENT table stores the payment transaction details of customers.

Attribute	Description
Payment_ID	Unique payment identifier
Booking_ID	Booking reference
Payment_Date	Date of transaction
Payment_Mode	UPI / Card / Net Banking / Cash
Payment_Status	Success / Failed / Pending
Transaction_Amount	Paid amount

SQL Query

```
SQL> CREATE TABLE PAYMENT(  
2     Payment_ID VARCHAR2(10) PRIMARY KEY,  
3     Booking_ID VARCHAR2(10),  
4     Payment_Date DATE,  
5     Payment_Mode VARCHAR2(20),  
6     Payment_Status VARCHAR2(15),  
7     Transaction_Amount NUMBER(10,2),  
8     CONSTRAINT FK_PAYMENT_BOOKING FOREIGN KEY (Booking_ID) REFERENCES BOOKINGS(Booking_ID)  
9 );
```

Table created.

3. Store Payment Details

Sample payment records are inserted into the PAYMENT table. These records are used to test payment status, payment methods, revenue, and customer payment reports.

```
SQL> INSERT INTO PAYMENT VALUES('P001','B001',TO_DATE('15-09-2026','DD-MM-YYYY'),'UPI','Success',360);
1 row created.

SQL> INSERT INTO PAYMENT VALUES('P002','B002',TO_DATE('17-09-2026','DD-MM-YYYY'),'Card','Pending',300);
1 row created.

SQL> INSERT INTO PAYMENT VALUES('P003','B003',TO_DATE('19-09-2026','DD-MM-YYYY'),'UPI','Success',480);
1 row created.

SQL> INSERT INTO PAYMENT VALUES('P004','B004',TO_DATE('21-09-2026','DD-MM-YYYY'),'Cash','Success',400);
1 row created.

SQL> INSERT INTO PAYMENT VALUES('P005','B005',TO_DATE('23-09-2026','DD-MM-YYYY'),'Card','Failed',340);
1 row created.

SQL> INSERT INTO PAYMENT VALUES('P006','B006',TO_DATE('24-09-2026','DD-MM-YYYY'),'Net Banking','Success',440);
1 row created.

SQL> INSERT INTO PAYMENT VALUES('P007','B007',TO_DATE('25-09-2026','DD-MM-YYYY'),'UPI','Success',320);
1 row created.

SQL> INSERT INTO PAYMENT VALUES('P008','B008',TO_DATE('26-09-2026','DD-MM-YYYY'),'Card','Pending',360);
1 row created.

SQL> INSERT INTO PAYMENT VALUES('P009','B009',TO_DATE('27-09-2026','DD-MM-YYYY'),'UPI','Success',400);
1 row created.

SQL> INSERT INTO PAYMENT VALUES('P010','B010',TO_DATE('28-09-2026','DD-MM-YYYY'),'Cash','Failed',340);
1 row created.
```

```
SQL> select * from payment;

PAYMENT_ID BOOKING_ID PAYMENT_D PAYMENT_MODE PAYMENT_STATUS
TRANSACTION_AMOUNT
-----
P001      B001      15-SEP-26 UPI      Success
360
P002      B002      17-SEP-26 Card      Pending
300
P003      B003      19-SEP-26 UPI      Success
480

PAYMENT_ID BOOKING_ID PAYMENT_D PAYMENT_MODE PAYMENT_STATUS
TRANSACTION_AMOUNT
-----
P004      B004      21-SEP-26 Cash      Success
400
P005      B005      23-SEP-26 Card      Failed
340
P006      B006      24-SEP-26 Net Banking Success
440

PAYMENT_ID BOOKING_ID PAYMENT_D PAYMENT_MODE PAYMENT_STATUS
TRANSACTION_AMOUNT
-----
P007      B007      25-SEP-26 UPI      Success
320
P008      B008      26-SEP-26 Card      Pending
360
```

4. Manage Successful and Failed Transactions:

The payment status is used to identify successful, failed, and pending transactions. SQL queries are used to display successful payments, find failed transactions, and count payments according to their status.

- **Display All Successful Payments:**

```
SQL> select * from payment where payment_status = 'Success';
```

PAYMENT_ID	BOOKING_ID	PAYMENT_D	PAYMENT_MODE	PAYMENT_STATUS
P001	B001	15-SEP-26	UPI	Success
	360			
P003	B003	19-SEP-26	UPI	Success
	480			
P004	B004	21-SEP-26	Cash	Success
	400			

PAYMENT_ID	BOOKING_ID	PAYMENT_D	PAYMENT_MODE	PAYMENT_STATUS
P006	B006	24-SEP-26	Net Banking	Success
	440			
P007	B007	25-SEP-26	UPI	Success
	320			
P009	B009	27-SEP-26	UPI	Success
	400			

6 rows selected.

- **Find Failed Transactions**

```
SQL> select * from payment where payment_status = 'Failed';
```

PAYMENT_ID	BOOKING_ID	PAYMENT_D	PAYMENT_MODE	PAYMENT_STATUS
P005	B005	23-SEP-26	Card	Failed
	340			
P010	B010	28-SEP-26	Cash	Failed
	340			

- **Count Successful and Failed Payments**

```
SQL> SELECT Payment_Status,COUNT(*) AS Total_Payments FROM PAYMENT GROUP BY Payment_Status;
```

PAYMENT_STATUS	TOTAL_PAYMENTS
Failed	2
Pending	2
Success	6

5. Analyze Payment Methods Used by Customers

Payment analysis helps identify which payment methods are commonly used by customers.

Report 1: Payment Mode Analysis

- Number of Transactions Using UPI

```
SQL> SELECT COUNT(*) AS UPI_TRANSACTIONS FROM PAYMENT WHERE Payment_Mode='UPI';
```

UPI_TRANSACTIONS
4

- Number of Card Payments

```
SQL> SELECT COUNT(*) AS CARD_TRANSACTIONS FROM PAYMENT WHERE Payment_Mode='Card';
```

CARD_TRANSACTIONS
3

- Most Preferred Payment Method

```
SQL> SELECT Payment_Mode,COUNT(*) AS TOTAL FROM PAYMENT GROUP BY Payment_Mode ORDER BY TOTAL DESC;
```

PAYMENT_MODE	TOTAL
UPI	4
Card	3
Cash	2
Net Banking	1

```
SQL> SELECT Payment_Mode,TRANSACTION_COUNT
2 FROM(
3 SELECT Payment_Mode,COUNT(*) AS TRANSACTION_COUNT
4 FROM PAYMENT
5 GROUP BY Payment_Mode
6 ORDER BY TRANSACTION_COUNT DESC
7 )
8 WHERE ROWNUM=1;
```

PAYMENT_MODE	TRANSACTION_COUNT
UPI	4

6. Report 2: Revenue Analysis

The Revenue Analysis report finds the total revenue, revenue generated through each payment method, and average transaction amount.

- Total Revenue Generated

```
SQL> SELECT SUM(Transaction_Amount) AS TOTAL_REVENUE
2  FROM PAYMENT
3  WHERE Payment_Status='Success';
```

```
TOTAL_REVENUE
-----
                2400
```

- Revenue by Payment Method

```
SQL> SELECT Payment_Mode,SUM(Transaction_Amount) AS REVENUE
2  FROM PAYMENT
3  WHERE Payment_Status='Success'
4  GROUP BY Payment_Mode;
```

```
PAYMENT_MODE          REVENUE
-----
Net Banking            440
Cash                   400
UPI                    1560
```

- Average Transaction Amount

```
SQL> SELECT AVG(Transaction_Amount) AS AVERAGE_TRANSACTION
2  FROM PAYMENT
3  WHERE Payment_Status='Success';
```

```
AVERAGE_TRANSACTION
-----
                  400
```

7. Report 3: Customer Payment History

The Customer Payment History report displays the customer name, booking ID, payment mode, amount, and payment status.

It displays:

- Customer Name
- Booking ID
- Payment Mode

- Transaction Amount
- Payment Status

```
SQL> SELECT C.FIRST_NAME || ' ' || C.LAST_NAME AS CUSTOMER_NAME,
2         P.BOOKING_ID,
3         P.PAYMENT_MODE,
4         P.TRANSACTION_AMOUNT,
5         P.PAYMENT_STATUS
6 FROM CUSTOMER C
7 JOIN BOOKINGS B
8 ON C.CUSTOMER_ID = B.CUSTOMER_ID
9 JOIN PAYMENT P
10 ON B.BOOKING_ID = P.BOOKING_ID;
```

CUSTOMER_NAME	BOOKING_ID
Akshaya Anantharaj UPI	B001
Divya Sri Card	B002
Vimal Krishna UPI	B003
Sri Vatsan Cash	B004
Pooja Shree Card	B005
Reena Vijay Net Banking	B006

CUSTOMER_NAME	BOOKING_ID
Amala Arun UPI	B007
Joseph Vijay Card	B008
Roselin Mary UPI	B009
Evangeline Sheeba Cash	B010

10 rows selected.

8. Payment Analysis:

The payment analysis helps the movie ticket booking system understand customer payment behaviour. It can be used to identify the most commonly used payment method, successful and failed transactions, total revenue, and customer payment history.

```
SQL> SELECT Payment_Mode,  
2         COUNT(*) AS TRANSACTION_COUNT,  
3         SUM(Transaction_Amount) AS TOTAL_REVENUE,  
4         ROUND(AVG(Transaction_Amount),2) AS AVG_TRANSACTION  
5 FROM PAYMENT  
6 WHERE Payment_Status='Success'  
7 GROUP BY Payment_Mode  
8 ORDER BY TOTAL_REVENUE DESC;
```

PAYMENT_MODE	TRANSACTION_COUNT	TOTAL_REVENUE	AVG_TRANSACTION
UPI	4	1560	390
Net Banking	1	440	440
Cash	1	400	400

9. Conclusion:

The Payment Transaction Management System was successfully implemented for the Movie Ticket Booking Management System. The PAYMENT table stores payment details and is connected to the BOOKINGS table using a foreign key. Payment transactions can be classified as successful, failed, or pending. SQL queries are used to analyze payment methods, revenue, and customer payment history. This helps the movie booking system manage payment information efficiently and generate useful business reports.